Textora Bulk SMS & API Gateway

Detailed Architectural, Pricing & Technical Specifications

---

1. System Overview

Textora is a high-speed, enterprise-grade Bulk SMS broadcast platform and developer API gateway. It operates as a dual-channel retail SMS management platform:

1. **Consumer Portal (Telegram & WhatsApp Bots)**: A conversational interface that enables individuals and business owners to upload contact lists, compose messages, customize sender identities, and execute bulk SMS campaigns on a pay-as-you-go basis. Textora is fully integrated and accessible on both **Telegram** and **WhatsApp**.
2. **Developer API (REST Gateway)**: A lightweight, developer-focused API that allows external systems to integrate SMS capabilities (such as One-Time Passwords, alerts, and customer notifications) directly into their custom backend codebases using a prepaid balance model.

Textora includes built-in concurrency protection, local persistent transaction locking, and a self-healing background worker that recovers from server crashes without dropping messages or double-billing clients.

---

2. Directory & Component Structure

The following layout describes the physical structure of the Textora application and the technical responsibility of each component:

```
textora/
├── index.js                    # Root entry point & dependency injection container
├── api-developers.json         # Local JSON store: developer profiles, balances, & keys
├── api-usage-log.json          # Local JSON store: developer API log entries
├── sender-ids.json             # Local JSON store: approved sender names per user ID
├── pending-sender-id-sessions.json  # Local JSON store: payment sessions for custom sender IDs
├── package.json                # Project manifests & dependencies
├── src/
│   ├── botHandlers.js          # Telegram & WhatsApp command/interaction flows
│   ├── config.js               # Environment verification & package price matrices
│   ├── contacts.js             # Number parser, sanitization, & spreadsheet extractor
│   ├── lockManager.js          # Local file-based concurrent thread locking controller
│   ├── paymentFlow.js          # Paystack fulfillment & SMS broadcast engine
│   ├── pricing.js              # SMS character counting, segment counting, & tiered pricing
│   ├── recoveryWorker.js       # Safe cron recovery worker for interrupted jobs
│   └── stores/                 # JSON database accessor classes
│       ├── apiDevelopers.js    # Read, write, & debit operations for developers
│       ├── apiLogs.js          # Read & write operations for API logs
│       ├── paymentSessions.js  # Active payment session status mapping
│       └── senderIds.js        # Approved sender names register
└── test/                       # Integration & unit test suites
    ├── apiEndpoints.test.js    # API endpoint server integration tests
    └── bulk-sms.test.js        # Bot campaign pricing & flow unit tests
```

---

3. Commercial & Pricing Specifications

Textora implements a dynamic tiered billing model separating local destinations (Ghana +233 networks) from all other international destinations.

3.1. General Rate Sheet

The following rates are charged per SMS segment (160 characters for GSM-7 encoding, 70 characters for Unicode encoding):

Destination	Bot User Campaign Rate	Developer API Rate
Local (Ghana)	Configured Selling Price (Default: **GHS 0.06**)	**GHS 0.05** (Deducts 1.0 credit)
International	Flat Rate: **GHS 0.08**	**GHS 0.08** (Deducts 1.6 credits)

3.2. Developer Prepaid Credit Packages

Developers purchase prepaid credits in fixed GHS bundles. Credits are stored in their profile and debited dynamically per segment:

- Local SMS segment: **Deducts 1.0 credit**.
- International SMS segment: **Deducts 1.6 credits** (reflecting GHS 0.08 value relative to the base credit cost of GHS 0.05).

GHS Price	Total Credits Purchased	Local SMS Yield (Segments)	International SMS Yield (Segments)
GHS 20.00	400	400	250
GHS 50.00	1,000	1,000	625

GHS 100.00	2,000	2,000	1,250
GHS 200.00	4,000	4,000	2,500
GHS 500.00	10,000	10,000	6,250
GHS 1,000.00	20,000	20,000	12,500
GHS 2,000.00	40,000	40,000	25,000
GHS 5,000.00	100,000	100,000	62,500

3.3. Custom Sender ID Setup Fees

- **Flat Registration Fee**: A one-time setup fee (configured via CUSTOM_SENDER_ID_FEE_GHS, e.g., GHS 50.00) is charged when requesting a custom Sender ID (up to 11 alphanumeric characters).
- **Zero LOA Requirement**: To optimize setup speeds, **Textora does not require a Letter of Authorization (LOA)** under any circumstances. Users pay the registration fee and the Sender ID is approved and configured directly on the gateway by the platform administration.

---

4. Architectural Workflows

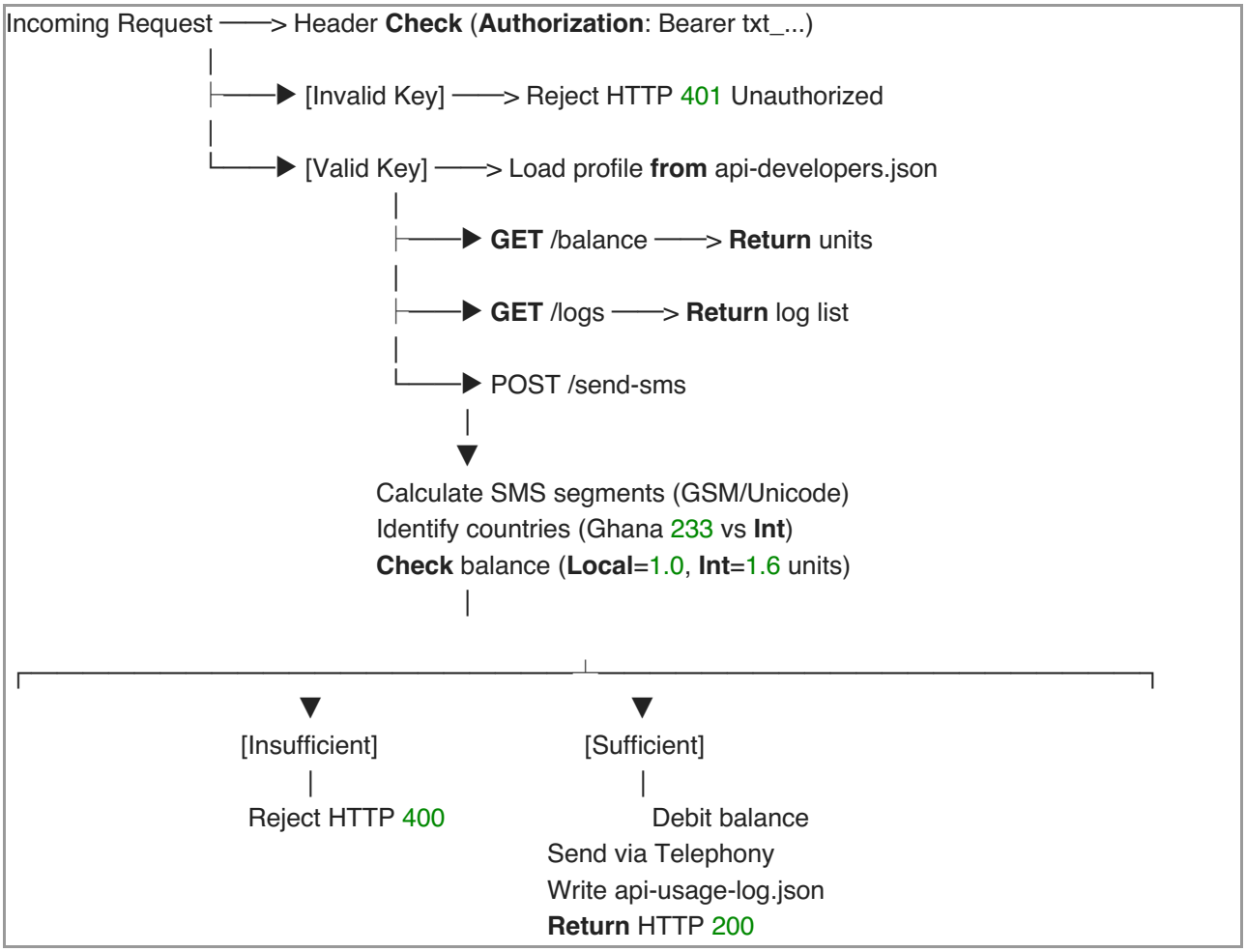
4.1. Paystack Webhook Verification & Processing

When a payment checkout is completed via Mobile Money or Card, Paystack hits the Textora HTTP server at the webhook path. The server processes this transaction securely using the following steps:

1. **Signature Validation**: Checks the x-paystack-signature header. The server computes the HMAC SHA256 signature of the raw request payload using the private PAYSTACK_SECRET_KEY and compares it to the header to prevent spoofing.
2. **De-duplication**: The webhook payload is parsed for the unique reference. If the reference is already marked as fulfilled in fulfilledPayments registry, the webhook terminates immediately to avoid double-processing.
3. **Session Retrieval**: Pulls the checkout session configuration from payment-sessions.json.
4. **Fulfillment Routing**:
    - If paymentPurpose is textora_bulk_sms: Executes the broadcast queue.
    - If paymentPurpose is sender_id_fee: Flags the Sender ID for manual registration.
    - If paymentPurpose is textora_api_recharge: Transitions the recharge to waiting_for_admin_approval and notifies the administrator.

4.2. Developer REST API Routing

The custom REST server accepts connections under the /api/v1/ prefix.



---

5. API Endpoints (REST Specification)

5.1. Get Balance

- **URL**: GET /api/v1/balance
- **Response**:

```
{
  "ok": true,
  "balance": 1450.00
}
```

5.2. List Verified Sender IDs

- **URL**: GET /api/v1/sender-ids
- **Response**:

```
{
  "ok": true,
  "senderIds": ["MYBRAND", "ALERT"]
}
```

5.3. Send SMS

- **URL**: POST /api/v1/send-sms
- **Request Body**:

```
{
  "to": ["233201234567", "1234567890"],
  "message": "Welcome to Textora. Your login verification code is 8876.",
  "senderId": "MYBRAND"
}
```

- **Response JSON**:

```
{
  "ok": true,
  "total_units_debited": 2.6,
  "remaining_balance": 1447.40,
  "results": [
    { "recipient": "233201234567", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } },
    { "recipient": "1234567890", "status": "success", "data": { "code": "ok", "message": "Submitted successfully" } }
  ]
}
```

5.4. Fetch API Logs

- **URL**: GET /api/v1/logs
- **Response**:

```
{
  "ok": true,
  "logs": [
    {
      "userId": "12345678",
      "timestamp": "2026-08-14T09:45:00.000Z",
      "message": "Welcome to Textora. Your login verification code is 8876.",
      "recipientsCount": 2,
      "unitsDebited": 2.6,
      "remainingBalance": 1447.40,
      "status": "success"
    }
  ]
}
```

---

6. Concurrency Protection & Recovery

6.1. Local File Transaction Locking (lockManager.js)

To avoid race conditions when multiple API calls or webhooks hit the server at the same millisecond, Textora uses a locking mechanism:

- Before reading/writing any shared JSON store (e.g. api-developers.json or payment-sessions.json), the thread must acquire a unique file lock (locks/<lockName>.lock).

- If a lock file exists, other threads wait, checking at a regular interval (retryIntervalMs = 50ms) until it is released or expires (lockExpiryMs = 60s).

6.2. Crash Recovery Worker (recoveryWorker.js)

When bulk SMS campaigns are broadcasted, they might consist of tens of thousands of contacts. If the server crashes mid-broadcast:

- The system writes individual progress states into sms-log.json for each contact.
- When the server boots up, it starts the **Textora Recovery Worker**.
- The worker scans payment-sessions.json for any transactions marked as processing that have been active for more than 120 seconds.
- It iterates over the campaign contacts, checks sms-log.json to see which recipients have already received the message, and **continues the broadcast for the remaining contacts** without duplicating any messages.